Software Dependability

Introduction to Software Dependability

The Role of the Software

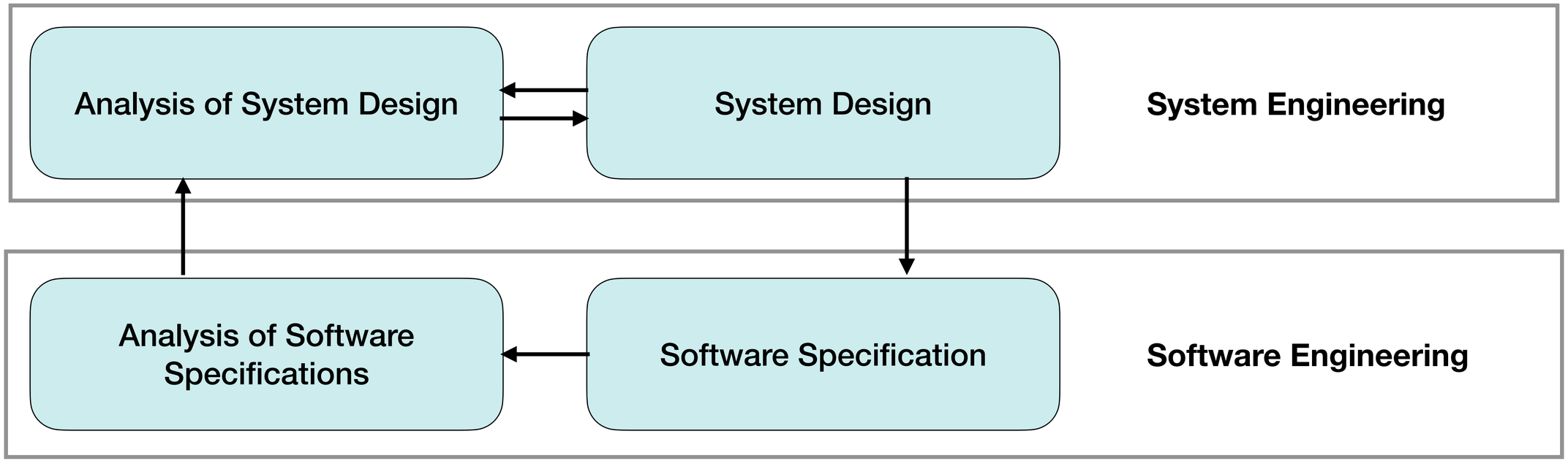
Software should perform the required function

Software should perform the required function **correctly**

Software might need to operate on a target platform whose design was influenced by the system's dependability goals.

Software often needs to take action when a hardware component fails

Systems and Software



What is a System?

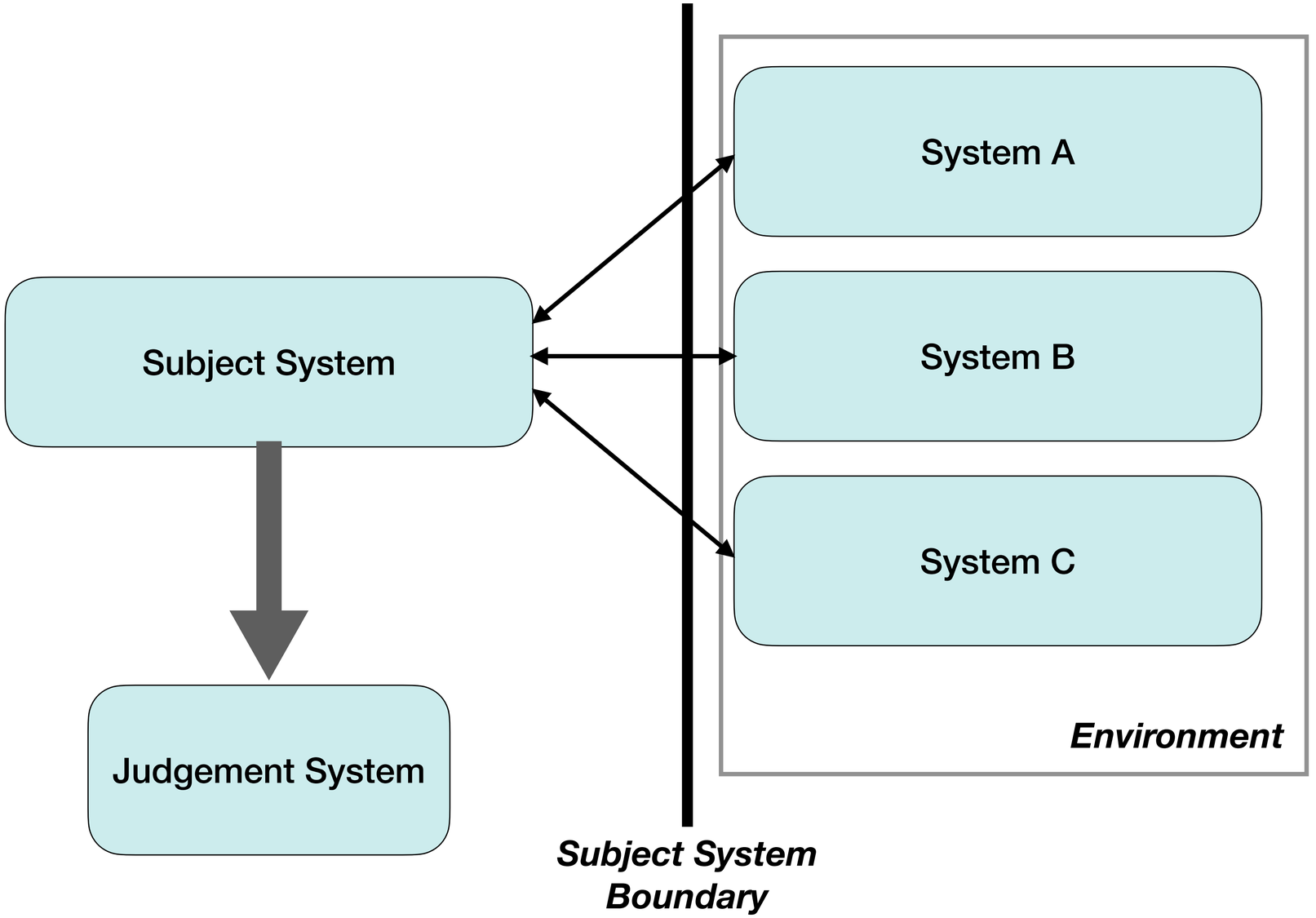
*“A **system** is an entity that interacts with other entities, i.e., other systems, including hardware, software, humans, and the physical world with its natural phenomena. These other systems are the environment of the given system.”*

*“The **system boundary** is the common frontier between the system and its environment.”*

“Systems are composed of systems, that are composed of systems, that are composed of systems, and so on.”

Avizienis, A., Laprie, J. C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. IEEE transactions on dependable and secure computing, 1(1), 11-33.

Relationship between Systems



Failures

A service is delivered correctly when it performs its functions.

A service failure, meaning a failure, is an event that happens when the delivered service differs from the correct service.

The effects of failures are serious!

We need clear definitions of how dependable our systems must be. We can't make them perfect! If we overlook something, the system might fail, and no one will care how well it worked.









Consequences of Failures

Some consequences are obvious; others are not.

Even minor systems can have severe results from failures, although in some cases, the impact appears limited to minor inconveniences. Failures in such non-critical systems **could affect related systems**.

For **security applications**, the consequences of failure are hard to evaluate but are **generally much greater than initially expected**.

Categories of Consequences

- Human injury or loss of life
- Damage to the environment
- Damage to or loss of equipment
- Damage or loss of data
- Financial loss by theft
- Financial loss through production of useless or defective products
- Financial loss through reduced capacity for production or service
- Loss of business reputation
- Loss of customer base
- Loss of jobs

Software Dependability - Definition

Dependability: *“[…] the trustworthiness of a computing system which allows reliance to be justifiably placed on the service it delivers […]”.*

IFIP 10.4 Working Group on Dependable Computing and Fault Tolerance

The dependability of a system is its ability to prevent service failures that are more frequent and more severe than acceptable. It reflects the extent of the user’s confidence when operating and interacting with it.

A system can be considered dependable if it operates without interruption, delivers the services that the stakeholders expect, does not adversely affect the system’s environment, and does not damage its data or the system itself.

Different ways to classify dependability, the stakeholder's perspective

The dependability of a system is *subjective*, i.e., it is not absolute but depends on the judgment of the stakeholders. For instance, what seems to be a failure to one stakeholder may represent an expected behavior to another.

Different ways to classify dependability, the dependability types

Functional vs. non-functional requirements.

If a stakeholder explicitly refers to dependability aspects, you have a functional dependability requirement, such as ensuring that all data is maintained on a local disk.

If you have requirements that specify dependability properties of the system, you have a non-functional dependability requirement, e.g., the probability of failure should be 0.001.

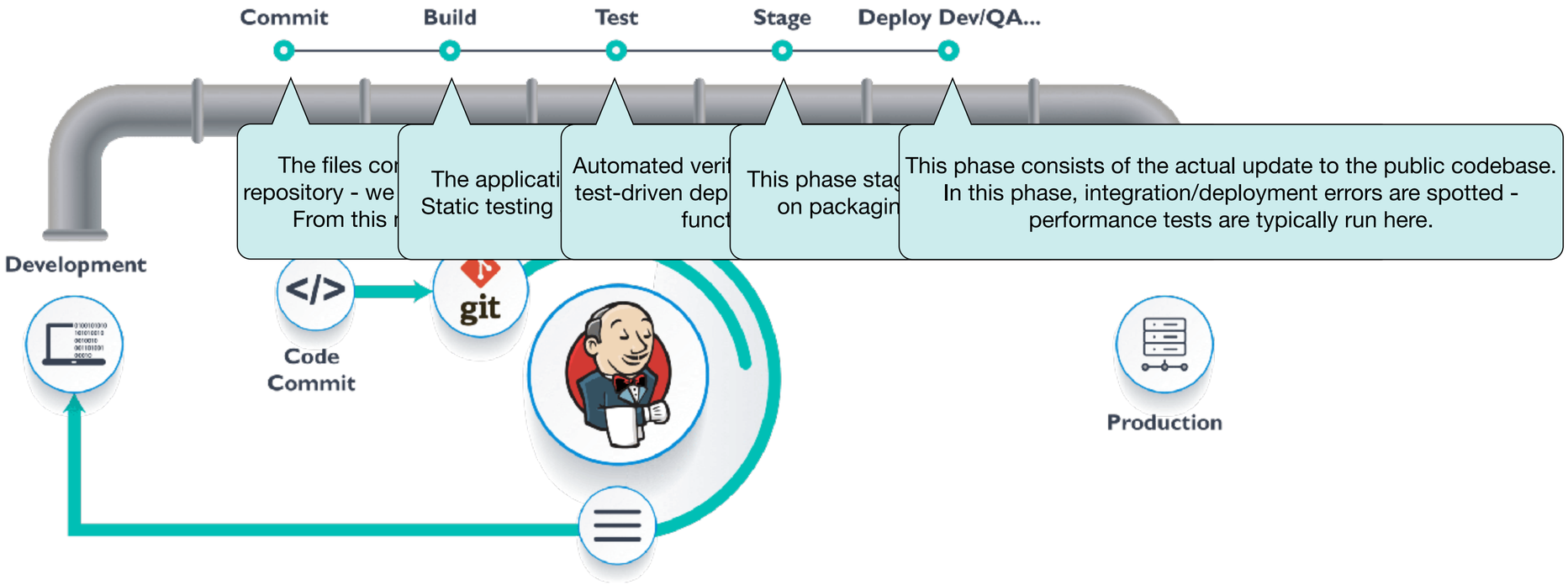
Different ways to classify dependability, the business perspective

Dependability issues should be considered alongside other business goals, as they can automatically lead to organizational problems.

For the sake of time, we will only touch the surface of the business perspective. However, I will upload additional material showing how dependability issues can lead to organizational troubles.

Software Dependability - A Comprehensive View

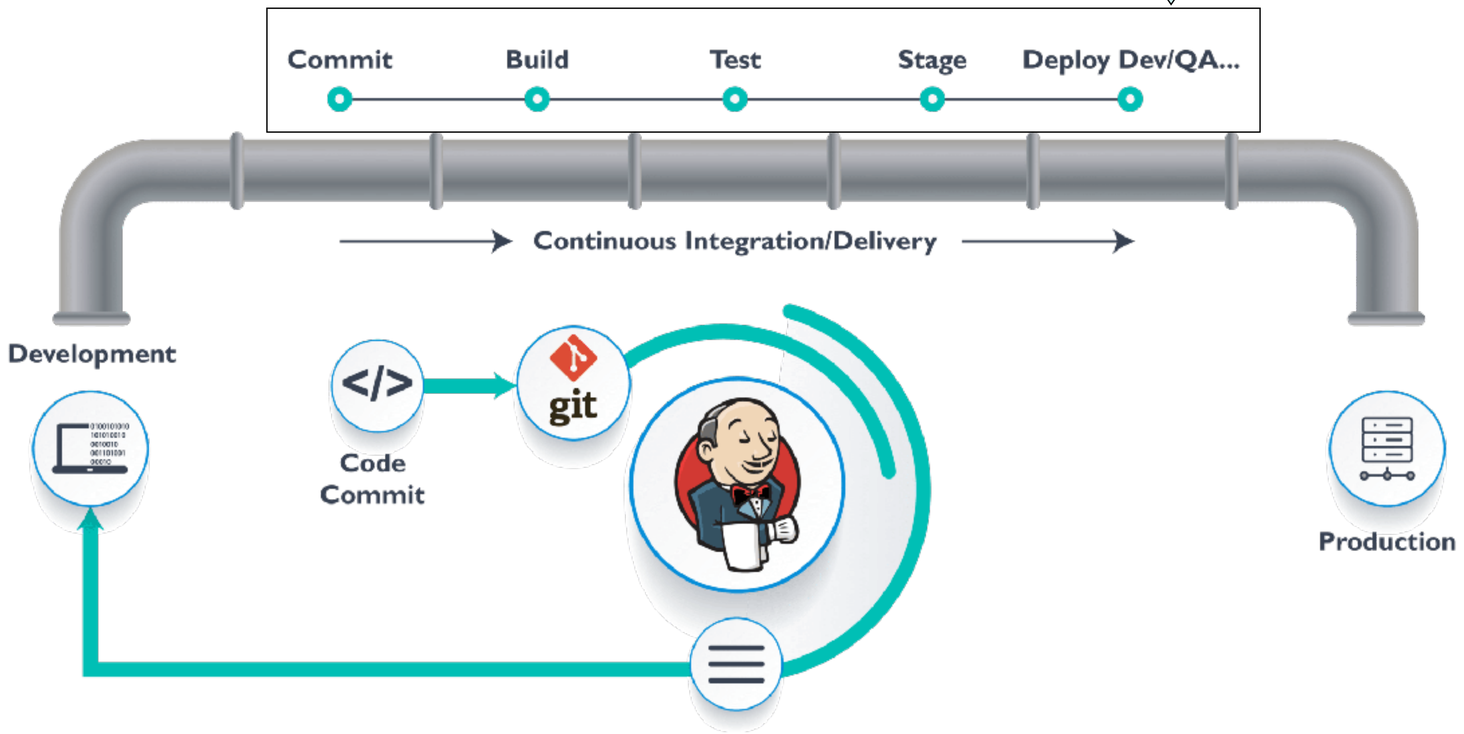
Indeed, dependability is an umbrella term to combine multiple aspects.



Software Dependability - A Comprehensive View

Indeed, dependability is an umbrella term to combine multiple aspects.

Verifying dependability is about the verification and validation of source code changes.

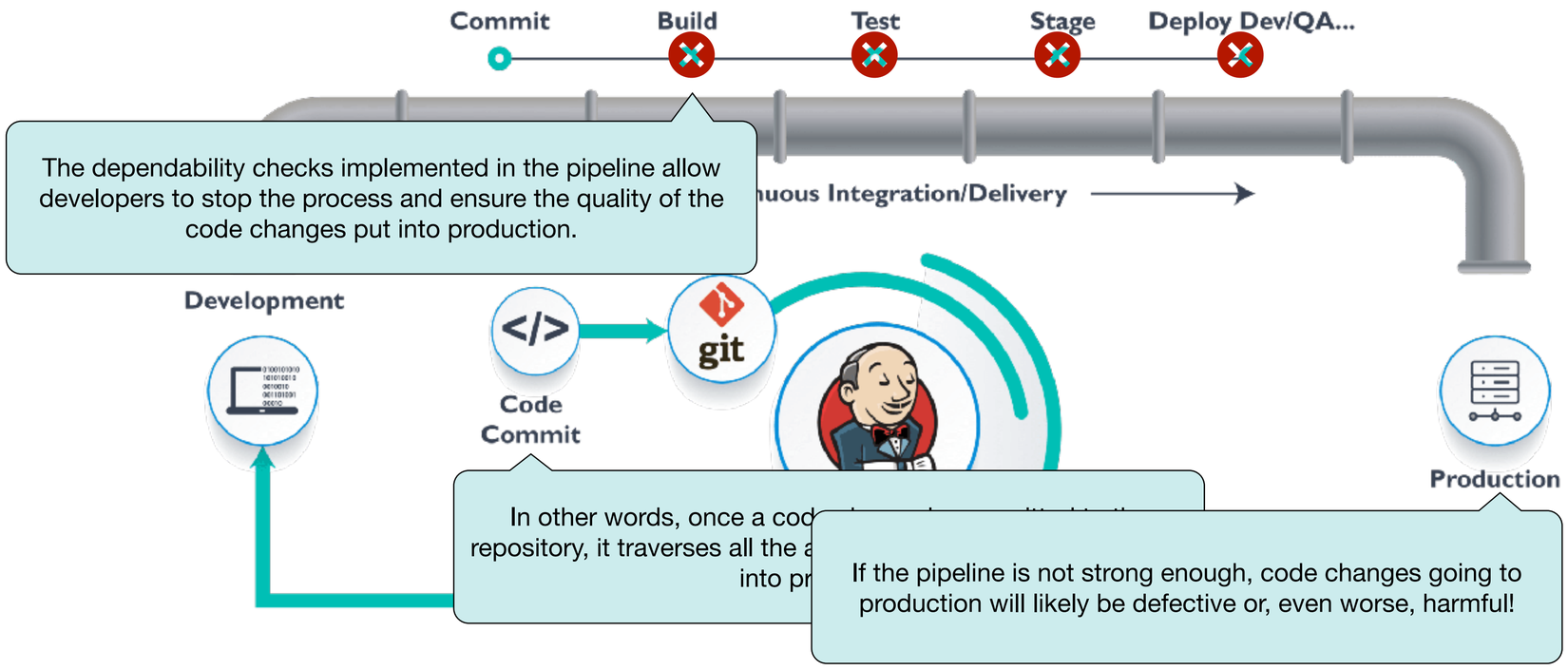


Continuous Integration and Delivery

Continuous Integration means integrating code into a shared repository and building/testing each change automatically, as early as possible - usually several times per day.

Continuous Delivery allows for releasing software to production anytime, often pushing changes to a staging system.

Continuous Deployment goes even further and pushes changes directly to production.

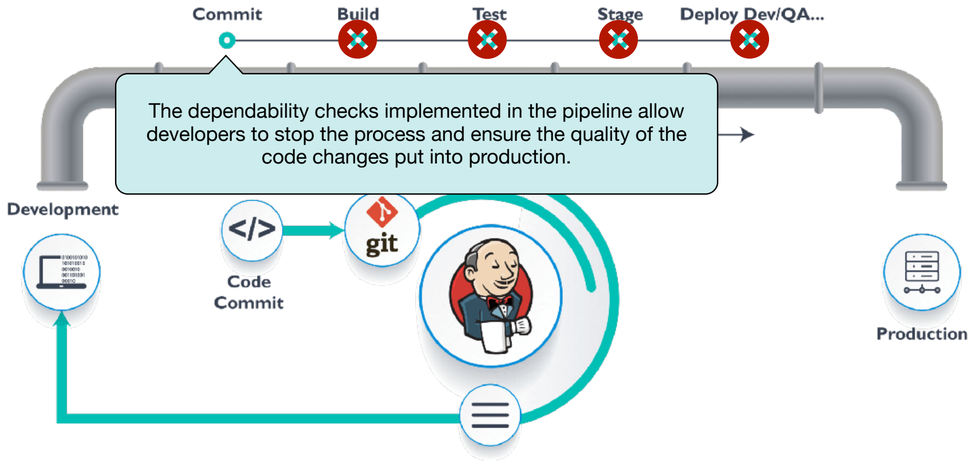


Continuous Integration and Delivery

Continuous Integration means integrating code into a shared repository and building/testing each change automatically, as early as possible - usually several times per day.

Continuous Delivery allows for releasing software to production anytime, often pushing changes to a staging system.

Continuous Deployment goes even further and pushes changes directly to production.



Most companies implement such a process and have fixed quality gates to reach!

Continuous Integration allows to:
1. Detect errors/vulnerabilities as quickly as possible —> Fix while fresh in mind!
2. Reduce integration errors —> Smaller problems are easier to digest!
3. Allow teams to develop faster and with more confidence to release good code.

The rules of CI/CD

(1) Failing fast. No matter what is wrong, it is essential to know it quickly!
- In such a scenario, code, and security inspections become crucial!
- Make tests easy to run, repeatable, and fast.
- Implement strategies to run subsets of tests (test case optimization).
- Track variations in code coverage to understand what needs to be tested more.
- Use liners that can spot design and security issues as soon as a new change is committed to the repository.

(2) Manage the output of CI. Be ready to take immediate contingency actions.
- Use the output of the CI process to understand what has created a failure in the build process.
- Verify the quality gates of your organization.
- Verify the outcome of test cases.

Applications with Dependability Requirements

Long-life applications. Applications with an operational life of several years. Typical examples are unmanned space flights and satellites. Typical requirements are a 95% or greater operational probability at the end of ten years.

Critical-computation applications. Applications that pose safety risks to people and businesses, such as aircraft, air traffic control systems, or infrastructure for controlling industrial plants, including nuclear or chemical plants.

Hardly maintainable applications. Applications in which the maintenance is costly and challenging to perform. Examples are remote processing systems in non-human regions (like the Antarctic continent). In this category, we can also mention the high-availability applications, like banking computing infrastructures.

NB: Most of these applications can be considered cyber-physical.

Therefore, we will also focus on some hardware aspects.

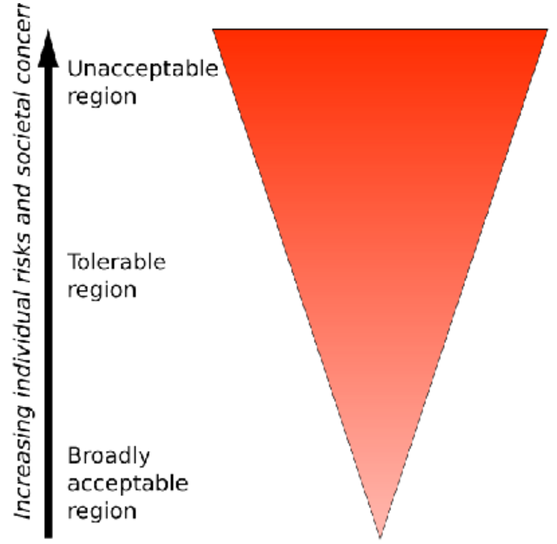
What Level of Risk is Acceptable?

As Low As Reasonably Practicable

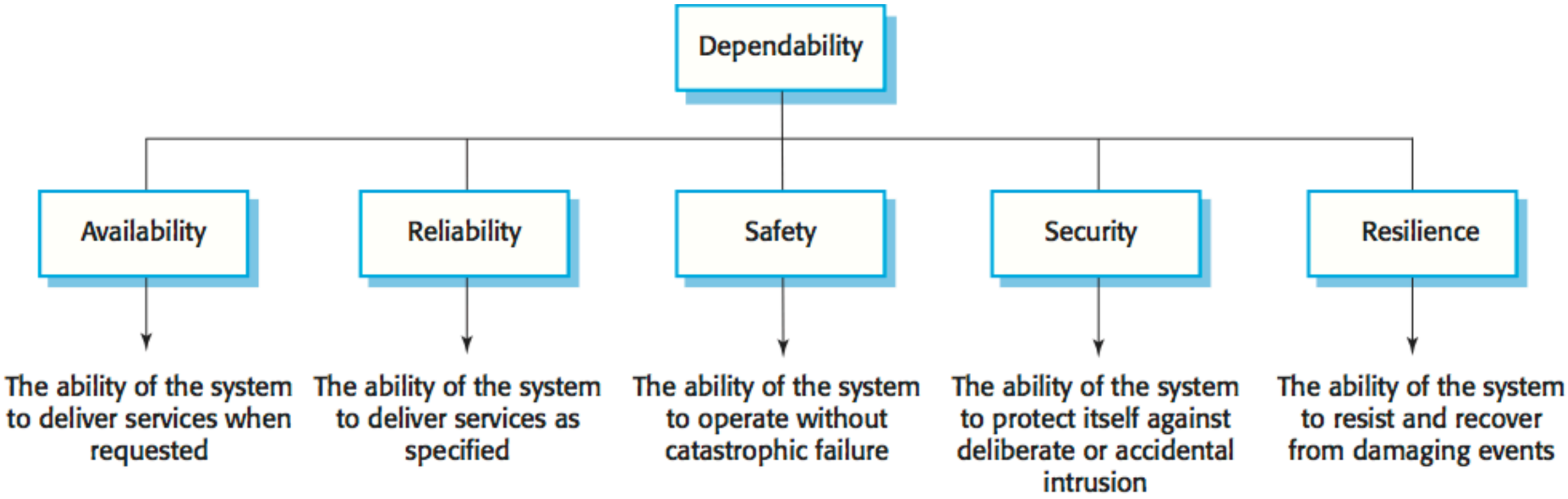
“As Low As Reasonably Practicable” (ALARP) is a principle used in risk management, especially in health and safety, to reduce risks to a level that is as low as reasonably achievable. It also applies to software dependability.

This idea balances the level of risk with the time, effort, and cost required to lower that risk further.

The goal is to minimize risk to a point where the cost and effort to decrease it more would be grossly disproportionate to the additional risk reduction.



A Closer Look to Dependability Attributes




Break